
DAWG'D: Disguised Adversarial Website Generation and Discriminator for Visual Web Agents

Andy Ye Andy Huynh Justin Zhou

Paul G. Allen School of Computer Science & Engineering
University of Washington

{andy132, ahuynh98, justzhou}@cs.washington.edu

Abstract

AI agents increasingly browse the web for us, and a growing class of them act purely from screenshots of a page—where everything they perceive are pixels that a malicious website can alter. We demonstrate how dangerous this is. By altering a small patch of the page (about 10% of its area) so subtly that a person would not notice, we steer the open screenshot agent MolmoWeb-4B at three escalating levels, and at varying success rates. We first force the web agent to answer a question incorrectly (success $\sim 60\%$); then make it return a believable lie—a mostly-true answer but a slightly altered fact, e.g. “4 people killed” becomes “14” (69%); and finally make it take the wrong action on the page. We present three major findings. First, what matters is the *amount* of the page we alter, not how it is arranged—one big patch and several small patches work equally well. Second, the obvious way to catch the believable-lie attack—checking whether the agent’s answer still “means the same thing”—misses 76% of the lies, because changing one fact barely shifts the sentence; a simple fact-by-fact check catches them instead. Third, we can reliably *disrupt* an agent’s action (90% of the time), but a small patch cannot force a *specific* attacker-chosen action. We also turn the attack into a defense: a small image classifier trained on our examples flags tampered screenshots nearly perfectly (0.9999 AUROC) on websites it has never seen, and the same signature transfers across all three attack types. We release our code and a labelled dataset of roughly 10,000 attacked screenshots.

1 Introduction

Screenshot-only web agents such as MolmoWeb [1] perceive a webpage purely as a rendered image and emit a browser action (click, type, navigate, or a message to the user). This makes them attractively general, requiring no DOM or accessibility tree, but it also means the entire attack surface is the pixel grid an attacker-controlled server renders. An adversary who can detect a bot and serve it a slightly perturbed page can, in principle, steer the agent with no change visible to a human.

We study this surface with white-box, pixel-space projected gradient descent (PGD) [7] against MolmoWeb-4B-Native, organized as three attack levels of increasing goal structure: Level 1 (L1) is untargeted answer drift, Level 2 (L2) is targeted “minimum-lie” misinformation, and Level 3 (L3) is action manipulation. We contribute a faithful differentiable bridge through MolmoWeb’s preprocessor for honest pixel-space attacks; a characterization of attack potency, patch geometry, and page robustness; the finding that targeted action-hijacking is far harder than targeted misinformation while untargeted disruption is easy; evidence that cosine similarity is inadequate for detecting plausible misinformation and a cascade that fixes it; and a noise-residual discriminator that detects the tampered screenshots, released with a labelled dataset.

2 Related Work

Web agents and benchmarks. There are many recent developments which enable visual web agents to interact with websites similar to how humans do. MolmoWeb [1] is an open screenshot-only web agent and the target of our attacks. As such frameworks have become popular, people have built benchmarks as well. WebArena [2] provides fully functional websites (e-commerce, forums, software, content management) for evaluating autonomous agents on long-horizon natural-language tasks, but is primarily text and accessibility-tree based; GPT-4 agents reach only 14.4% there versus 78.2% for humans. VisualWebArena [3] extends this to visually grounded tasks that require multimodal agents to reason over screenshots and images, making it the natural setting in which to inject our pixel attacks into realistic tasks and to evaluate a screenshot agent augmented with our discriminator (future work).

Attacks on VLM agents. As VLMs rise in popularity, people have learned to attack them. Recent work attacks vision–language agents with explicit on-page content such as pop-ups and injected text [4], which is highly effective but human-visible. WebInject [5] forces a screenshot agent to perform an attacker-specified action by adding a *whole-screen* adversarial perturbation to the rendered page (approximating the non-differentiable render-to-screenshot step with a learned network), reaching $\sim 97\%$ action-injection success across five agents.

Adversarial attacks and patches. L_∞ -bounded PGD [7] and momentum variants (MI-FGSM) [8] are standard first-order white-box attacks; localized adversarial patches [9] trade imperceptibility for spatial concentration, as in our threat model.

Relation to prior work. DAWG’D differs in three ways: (i) our perturbation is a *localized patch*, not a full-image perturbation, trading attack strength for a smaller, more disguised footprint; (ii) we obtain gradients through an *exact* differentiable replica of the model’s preprocessor rather than a learned approximation of the render step; and (iii) we go beyond actions to untargeted answer-drift (L1) and targeted plausible-misinformation (L2)—neither of which WebInject addresses—and add a trained detector. More broadly, Zhao et al. [6] show that PGD attacks transfer across large vision–language models; we complement the attack side with a concrete defense.

3 Method

3.1 Attack setup

Target and attacker. MolmoWeb-4B-Native takes a screenshot plus a task and outputs reasoning and one browser action. The attacker controls server-side rendering and may serve a perturbed page only to the agent, so human imperceptibility is a soft constraint. We perturb a patch covering $\sim 10\%$ of the page with L_∞ budget $\epsilon \in [12, 24]/255$.

Differentiable preprocessing bridge. PGD requires gradients through MolmoWeb’s non-differentiable preprocessor (bilinear resize, SigLIP normalization, 2×4 tiling plus a global thumbnail). We reimplement this path differentially and verify it matches the real preprocessor to within $0.5/255$ at every attack resolution, so gradients flow from pixels to patch.

Patch placement and optimization. We query MolmoWeb itself to point at the element answering the task and center the patch there, clipping to stay on target near page edges (Figure 1). For multiple patches, we paint over the previously chosen coordinates and re-query for the most relevant element such that patch selection is not biased. We then optimize the patch with momentum PGD (MI-FGSM), ~ 100 iterations, projecting to the ϵ -ball intersected with the patch mask each step. The loss is a teacher-forced cross-entropy $\mathcal{L}$ on the agent’s output tokens; each level (below) sets whether we *maximize* it (push the output away from clean) or *minimize* a masked version (pull the output toward a chosen target, with the mask restricting the gradient to the tokens we want to change).

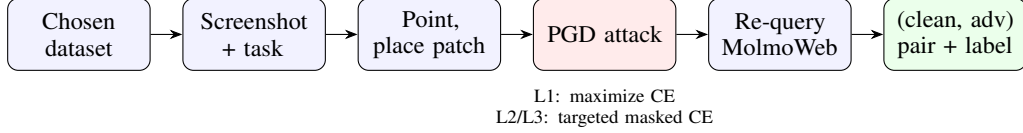


Figure 1: Data-generation pipeline, shared by all three attack levels. A clean screenshot from the chosen dataset (SyntheticQA for L1/L2, SyntheticTrajs for L3) is paired with a task; MolmoWeb points at the relevant element so we place the patch there; PGD perturbs the patch toward the level’s objective; we re-query the agent and store the (clean, adversarial) pair with its label.

3.2 Dataset generation

Each level specifies a *data source*, an *attack objective*, and a *success check* that decides whether the attack worked. Table 1 summarizes the resulting dataset and Figure 2 shows a representative pair from each level. We use a 10% total adversarial patch size on each website. For L1, we studied if dividing up this larger parts into multiple smaller areas matters.

L1: Untargeted answer drift. *Data:* real screenshots and ground-truth Q&A from MolmoWeb-SyntheticQA. *Attack:* maximize $\mathcal{L}$ on the clean-answer tokens, pushing the agent’s answer as far away as possible from correct. We test this with 10% sized adversarial patches, two 5% sized patches, and three 3.3% sized patches. All patches are positioned by prompting MolmoWeb with the question and asking it to click on the most relevant coordinates on the site to answer this question, then painting a adversarial patch over that region. *Success check:* we compare the adversarial answer to the clean answer with a semantic-equivalence test—we first unwrap the JSON action, then declare “same meaning” on an exact match, a token-subset match, or if MPNet [10] sentence-embedding cosine similarity is ≥ 0.75 . The attack succeeds if the adversarial answer no longer means the same as the clean answer. It is important to realize that we are assuming the webagent returns the right answer without any adversarial attack.

L2: Targeted minimum-lie. *Data:* SyntheticQA questions whose answer contains a perturbable fact. *Attack:* we build a target answer that corrupts exactly one fact (a yes/no flip, a number or year shift, or a dropped list item) and descend a masked CE toward it. The mask covers only the tokens that differ from the clean answer, so just the load-bearing fact carries gradient. We attack with only 10% sized patches with smart placement. *Success check:* we build a factual-drift cascade, ordered cheapest-first. Layer 0 is a regex set-difference over numbers and dates; Layer 2 is bidirectional natural-language inference [11] for contradictions and dropped qualifiers; sentence-embedding cosine is used to determine if the tampered answer is still plausibly correct (similar enough). The attack succeeds if the cascade detects drift, the answer remains plausible, and the output is coherent. The omission case—a dropped list item with identical numbers—is invisible to Layer 0 and to cosine, and is caught only by NLI.

L3: Action manipulation. *Data:* MolmoWeb-SyntheticTrajs, which pairs screenshots with the agent’s ground-truth browser action at each step. *Attack:* we use the same technique in L1, applied to the action JSON. Untargeted disruption maximizes $\mathcal{L}$ on the clean action, and targeted hijacking minimizes masked CE toward an attacker action (e.g. goto an attacker URL, or a redirected click). We attack with only 10% sized patches with smart placement. *Success check:* we parse the post-attack action; disruption succeeds if the action changes from the clean one, and targeted hijacking succeeds if the parsed action equals the attacker’s chosen target.

3.3 Training the discriminator

We present a discriminator which is able supplement visual web-agent queries to determine if a website has been attacked. Treating the released (clean, adversarial) pairs as supervision, we train a whole-image classifier that flags tampered screenshots. Each screenshot is letterboxed to 512×512 and passed through a fixed SRM high-pass residual front end [12]: localized high-frequency tampering is exactly what such steganalysis residuals surface, and the high-pass strips page content so the network keys on the patch’s high-frequency signature rather than the page’s identity. The

Table 1: Items in the DAWG dataset. L1/L2 attack MolmoWeb-SyntheticQA, while L3 uses MolmoWeb-SyntheticTrajs screenshots. Success is conditioned on a correct clean answer.

Level	Geometry / target	Pairs	Success
L1 single-10%	one 10% patch on the answer	2,522	63%
L1 two-5%	two 5% patches	1,284	57%
L1 three-3.3%	three 3.3% patches	1,227	58%
L2 minimum-lie	one corrupted fact	2,435	69% (target hit)
L3 disruption	untargeted action change	2,680	90%



Figure 2: Clean vs. adversarial pairs, zoomed to the patched region; the red box marks the bounded perturbation. The same small patch suffices to drift the answer (a), plant a one-word lie (b), or disrupt the action (c).

residual feeds an ImageNet-initialized EfficientNet-B0 [13] with a binary head, trained with cross-entropy. Positives are the L1 and L2 adversarial screenshots (6,002); negatives are clean screenshots whose page was never attacked (8,331), so an attacked page’s clean twin is never reused. We split *site-disjoint* (66 held-out sites) so the model must learn the perturbation, not the site.

4 Results

4.1 L1: potency, patch geometry, and what resists

We find that a 10% patch needs $\epsilon \approx 12$ to function, anything lower is too weak. We find above $\epsilon \approx 24$ it’s efficacy is the same but it becomes more visible, saturating at a $\sim 73\%$ ceiling.

We discovered that how the patch budget is arranged barely matters; the perturbed area does. A natural worry is that one large patch is conspicuous, so we compared splitting the same $\sim 10\%$ budget into one, two, or three patches across 210 pages and a swept ϵ (Figure 3). A single concentrated patch on the answer is best-or-tied in every condition, but the gaps between one 10% patch, two 5% patches, and three 3.3% patches all fall within Wilson 95% confidence intervals. Thus, we posit that total perturbed area, not its arrangement, is the operative variable, which lets an attacker fragment the budget into less conspicuous pieces at little measurable cost in success.

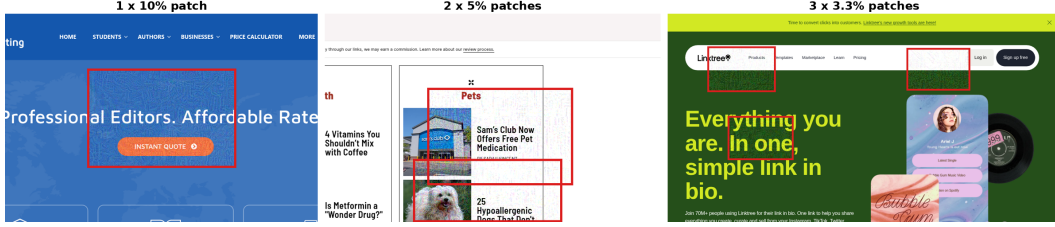


Figure 3: The three patch geometries at matched $\sim 10\%$ area, with the bounded patches outlined in red: a single 10% patch, two 5% patches, and three 3.3% patches placed on the top-1, top-2, and top-3 elements MolmoWeb points to. Total perturbed area is the same in each.

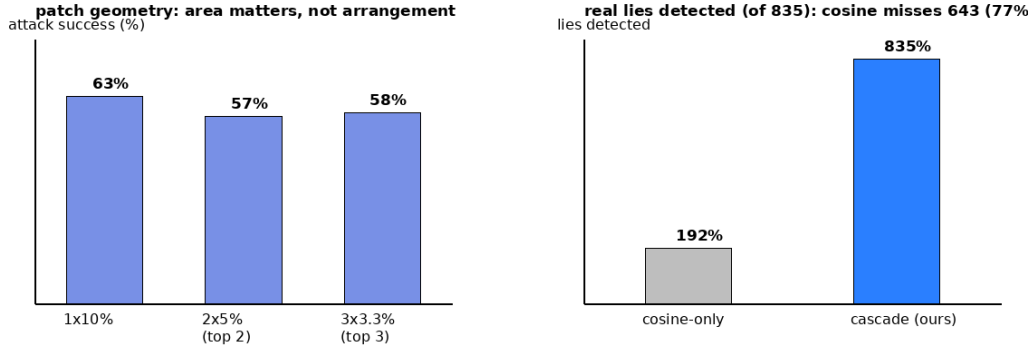


Figure 4: **Left:** attack success by patch geometry at matched $\sim 10\%$ area; one 10% patch, two 5% patches, and three 3.3% patches all fall within Wilson confidence intervals of one another, so total perturbed area rather than its arrangement drives success. **Right:** L2 detection over 2,435 pages; of the 2,082 real lies the cascade detects, a single cosine-similarity check recovers only 495, missing 1,587 (76%).

4.2 L2: plausible lies evade a cosine check

At scale, 69% of L2 attacks hit the exact target lie across 2,435 pages, and 1,965 satisfy the full success criterion of drift, plausibility, and coherence. The striking result is on the detection side. Using the cosine detector from L1, we are unable to detect if a plausible lie was generated. This is because the adversarial query returns a plausible lie, and to be plausible it must have similar cosine similarity. Over the 2,082 examples our cascade marks as real drift, a cosine-only check missed 1,587 of them—76% (Figure 4). Our factual-drift cascade closes this gap.

4.3 L3: untargeted disruption is easy, targeted hijacking is hard

The action attacks reveal a sharp asymmetry. Untargeted disruption succeeds 90% of the time across 2,680 attacked steps from 29 sites (Figure 2c). Targeted hijacking, however, fails to converge under our patch. We believe this is due to the *localized-patch budget*, not to web-agent robustness in general: WebInject [5] achieves $\sim 97\%$ targeted action injection on similar agents using a *whole-image* perturbation, which has far more degrees of freedom than our $\sim 10\%$ patch. Within the bounded-patch setting, then, we find that the more targetted of an attack, the more difficult it is to do. We find untargeted disruption stays feasible at every level.

4.4 The discriminator detects tampered screenshots

On the site-disjoint test split (66 held-out sites; 1,096 adversarial / 1,873 clean) the detector reaches 0.9999 held-out AUROC and 0.999 true-positive rate at 1% false-positive rate, with per-geometry AUROC of 1.0 across all three L1 geometries and L2. Two controls confirm this is genuine patch detection, not a content or pipeline shortcut. On matched pairs—the same page’s clean twin vs. its adversarial version, identical content and save pipeline—it still separates at 0.9996 AUROC (2% false

positives on clean twins). And a detector trained only on L1/L2 catches the never-seen L3 patches at 1.0 AUROC (100% caught, 0.2% false positives), so the signature generalizes across attack levels.

5 Discussion and Conclusion

DAWG'D shows that screenshot web agents are easy to mislead in their outputs and disrupt in their actions, but hard to steer into a specific chosen action under a bounded patch, and that attack success depends on perturbed area rather than arrangement. Two findings are defensively useful: detecting plausible misinformation needs more than embedding similarity, and the patches carry an obvious high-frequency fingerprint a noise-residual detector flags at near-perfect, attack-level-transferable AUROC. Natural next steps are end-to-end evaluation inside VisualWebArena [3] and adversarially fine-tuning MolmoWeb on the released pairs (mapping tampered screenshots to “no action”). The main caveat is adaptivity: an attacker who penalizes the residual signature could evade a fixed detector.

References

- [1] AllenAI (2026) MolmoWeb: Open Visual Web Agent and Open Data for the Open Web. *arXiv:2604.08516*.
- [2] Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., et al. (2023) WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv:2307.13854*.
- [3] Koh, J.Y., Lo, R., Jang, L., Duvvur, V., Lim, M.C., Huang, P.-Y., et al. (2024) VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. *arXiv:2401.13649*.
- [4] Zhang, Y., et al. (2024) Attacking Vision-Language Computer Agents via Pop-ups. *arXiv:2411.02391*.
- [5] Wang, X., Bloch, J., Shao, Z., Hu, Y., Zhou, S. & Gong, N.Z. (2025) WebInject: Prompt Injection Attack to Web Agents. *EMNLP 2025*; *arXiv:2505.11717*.
- [6] Zhao, Y., Pang, T., Du, C., Yang, X., Li, C., Cheung, N.-M. & Lin, M. (2023) On Evaluating Adversarial Robustness of Large Vision-Language Models. *NeurIPS*.
- [7] Madry, A., Makelov, A., Schmidt, L., Tsipras, D. & Vladu, A. (2018) Towards Deep Learning Models Resistant to Adversarial Attacks. *ICLR*.
- [8] Dong, Y., Liao, F., Pang, T., Su, H., Zhu, J., Hu, X. & Li, J. (2018) Boosting Adversarial Attacks with Momentum. *CVPR*.
- [9] Brown, T.B., Mané, D., Roy, A., Abadi, M. & Gilmer, J. (2017) Adversarial Patch. *NeurIPS Workshop*.
- [10] Song, K., Tan, X., Qin, T., Lu, J. & Liu, T.-Y. (2020) MPNet: Masked and Permuted Pre-training for Language Understanding. *NeurIPS*.
- [11] He, P., Liu, X., Gao, J. & Chen, W. (2021) DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *ICLR*.
- [12] Fridrich, J. & Kodovsky, J. (2012) Rich Models for Steganalysis of Digital Images. *IEEE TIFS*.
- [13] Tan, M. & Le, Q. (2019) EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML*.